

MGraph-AI Service Base: The Production-Ready Microservice Template

From "Infrastructure Later"
to "Production Day One".



The Hidden Cost of “We’ll Productionise Later”



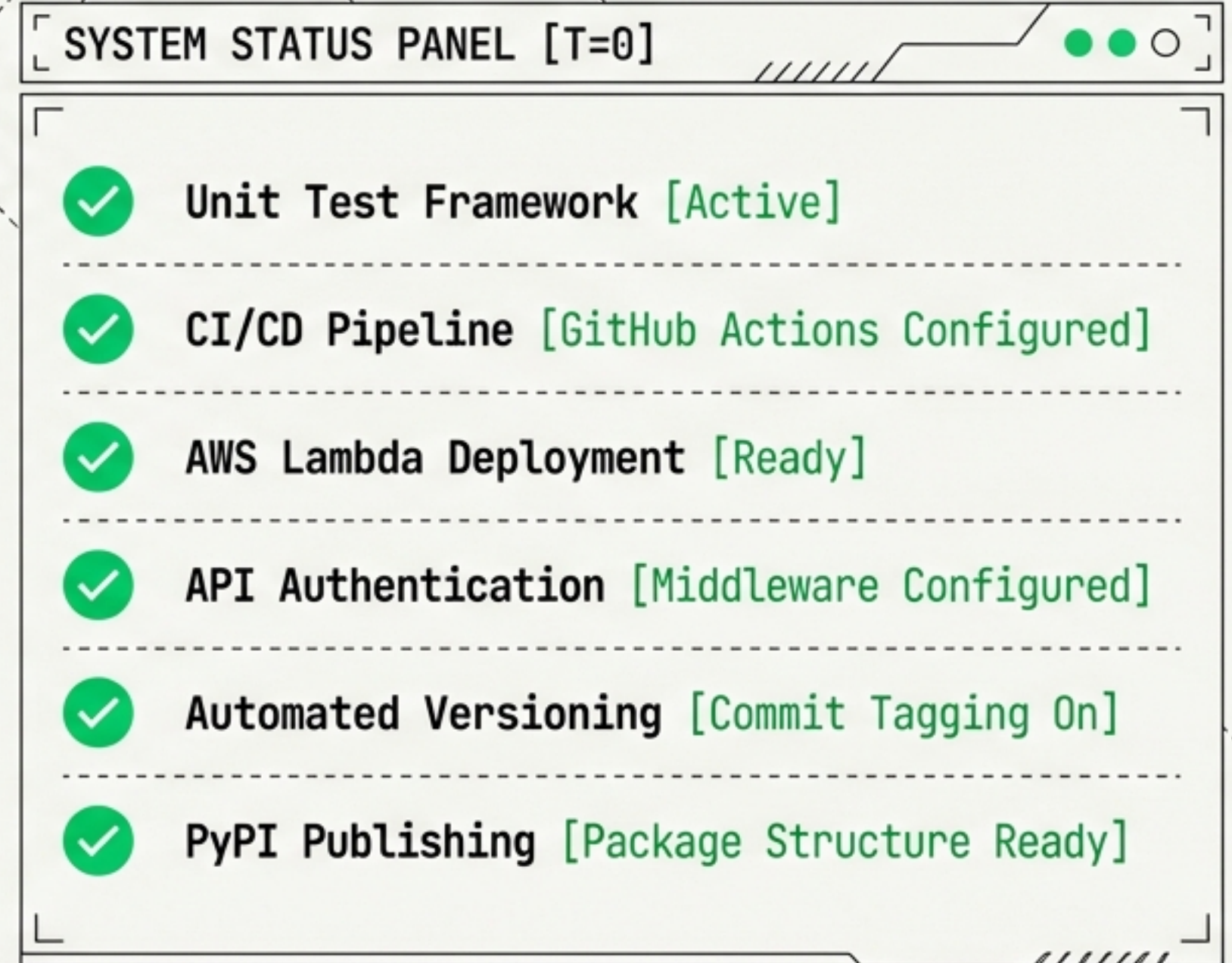
Most teams build the feature first, then struggle with deployment. The consequence is that technical debt accumulates in deployment processes, not just in the code itself.

- Inconsistent environments
- Missing test coverage
- ‘Works on my machine’ friction

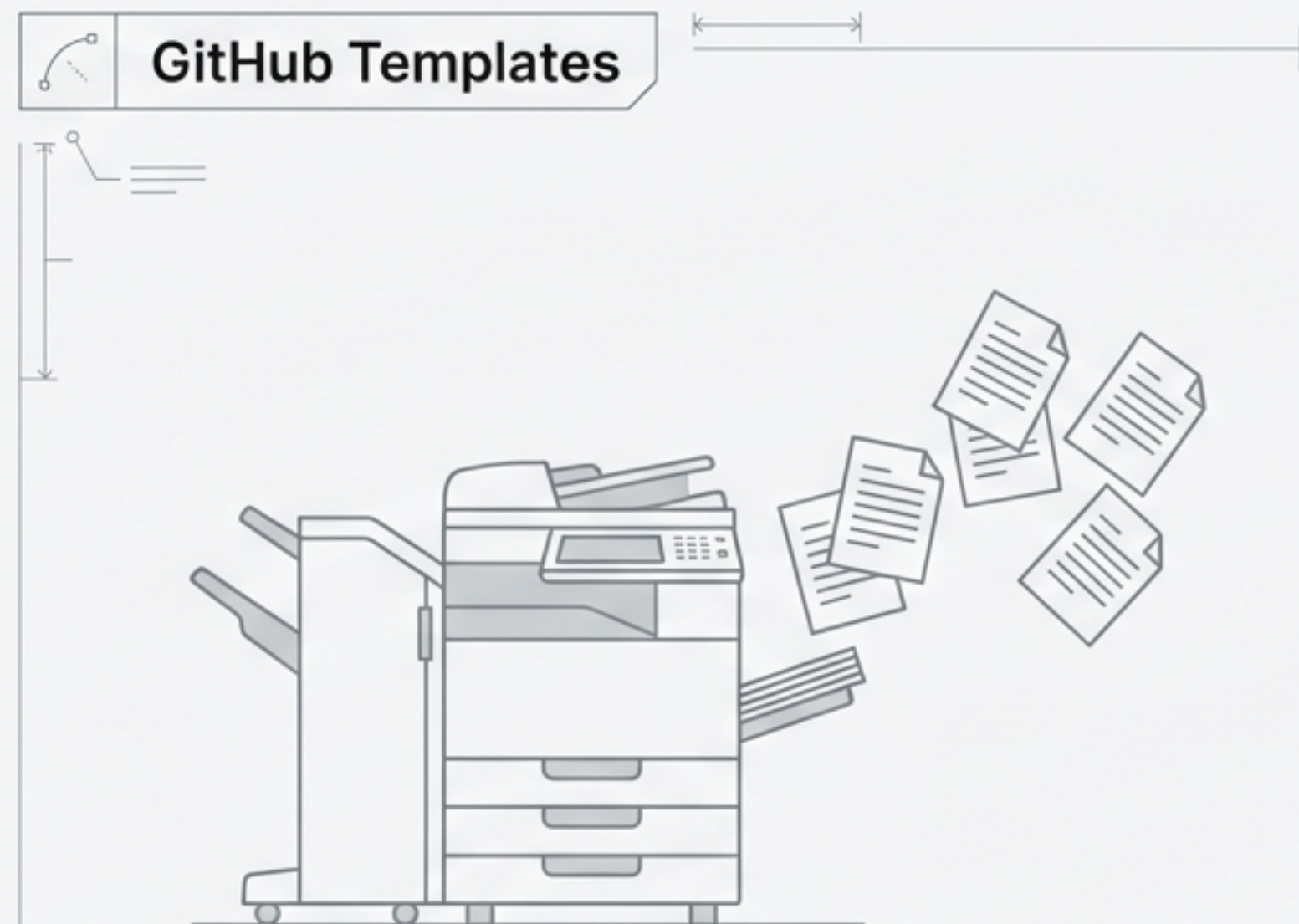
The Strategic Inversion: Infrastructure at T=0

What exists before the first line of business logic is written?

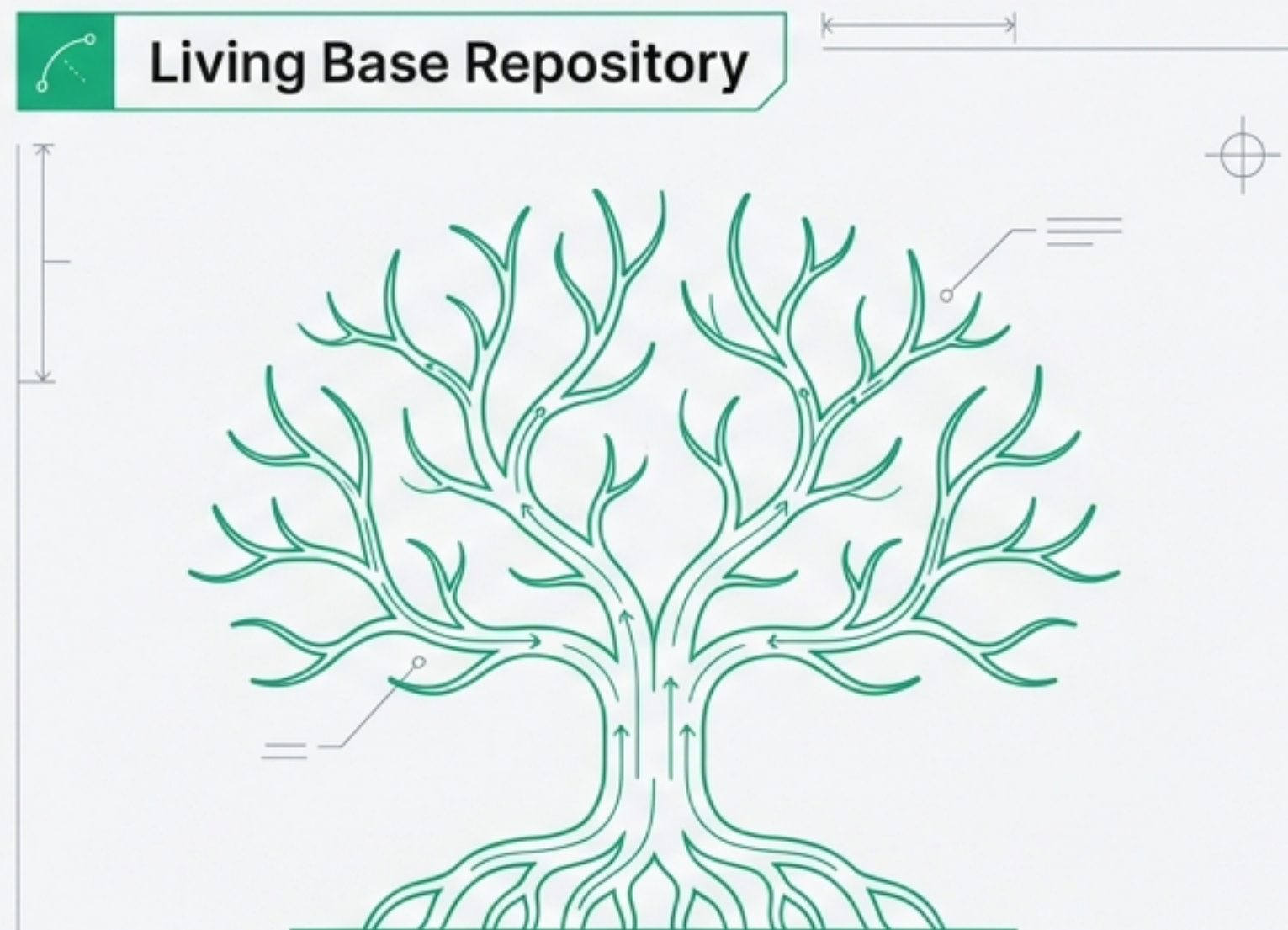
**Eliminate the gap
between “I have an
idea” and “I have a
production service”.**



Moving Beyond Static Templates



Static snapshot. Once copied, the link is broken. Bugs are discovered and fixed in isolation per project.



Biological inheritance. Upstream improvements flow down. A bug fixed in the base is propagated to all services instantly.

The Utility Framework Architecture

Business Logic

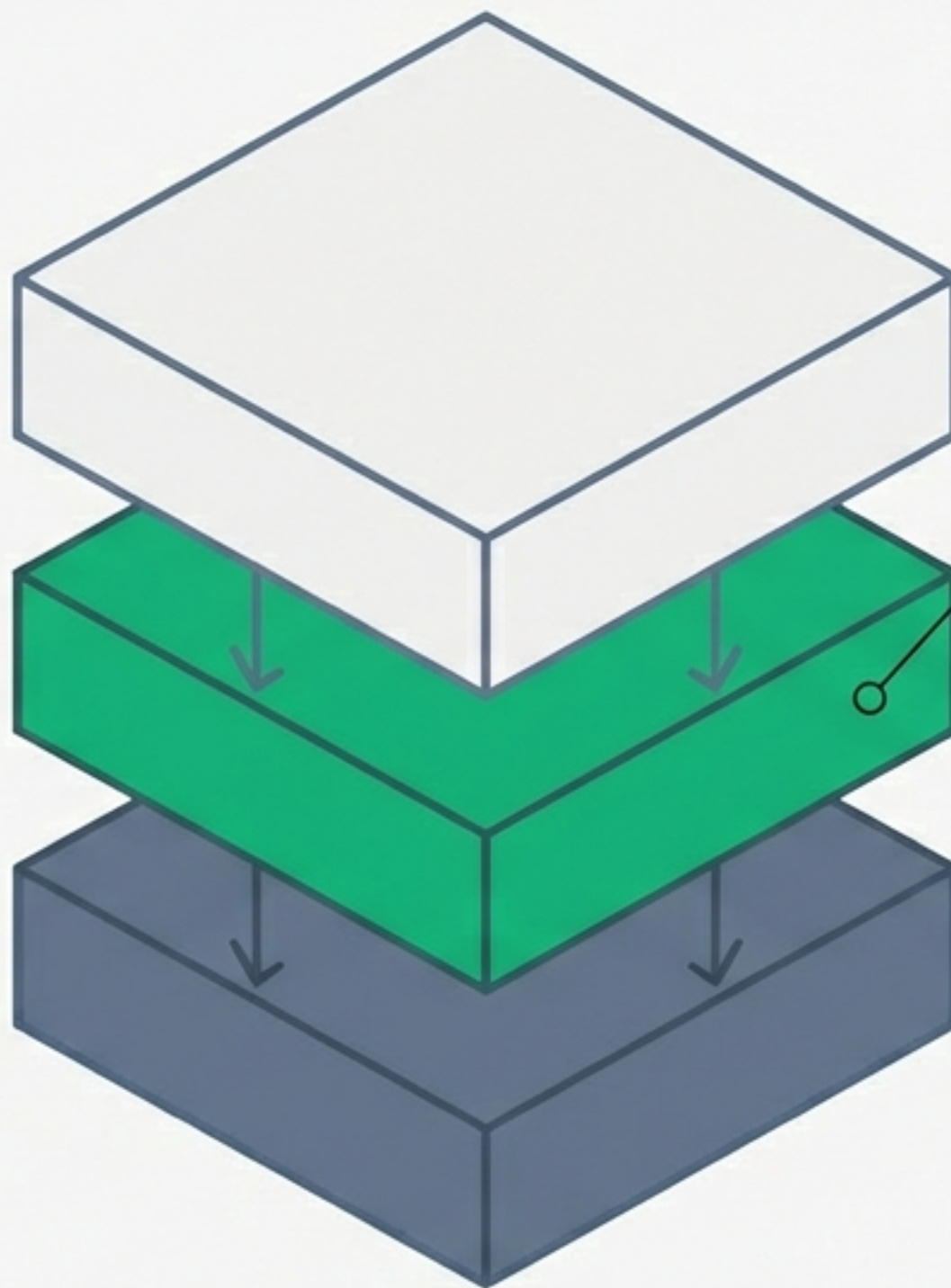
Service specific code

Type_Safe

Base classes, Runtime checking, Serialisation

osbot-aws

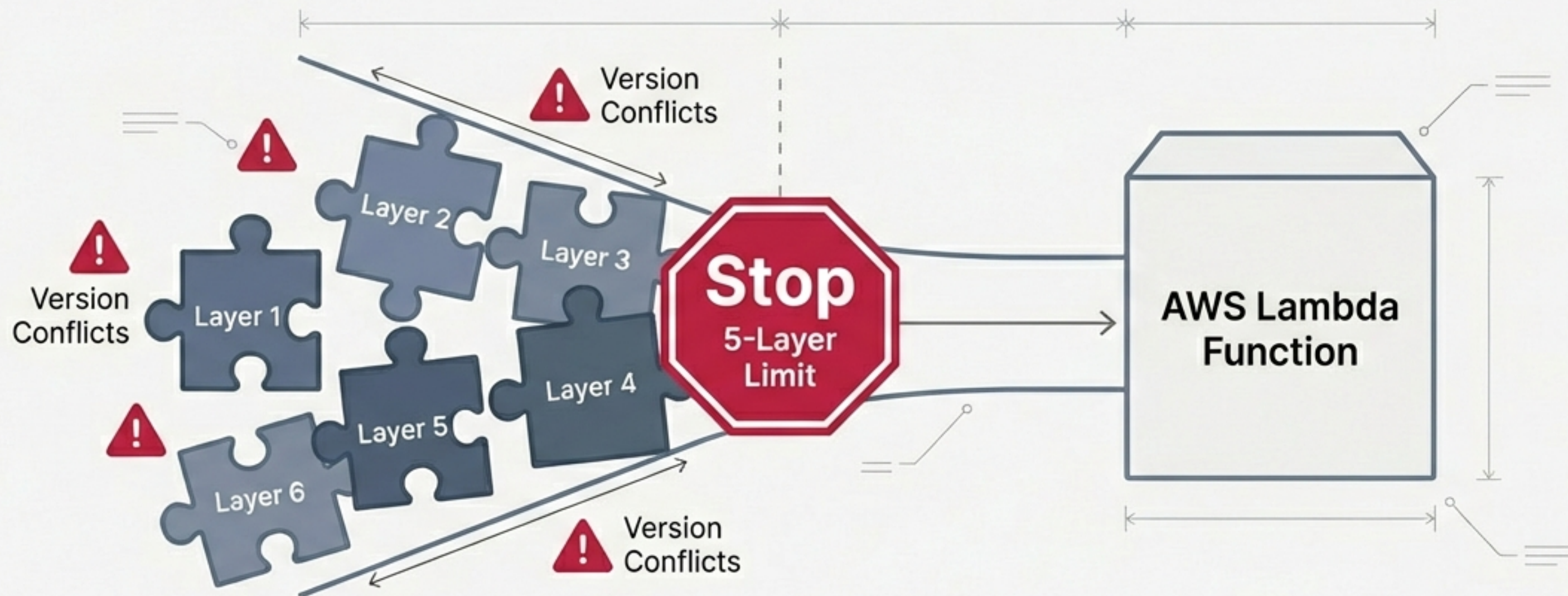
Cloud interaction primitives



Ensures consistent object patterns and automatic serialisation across the entire organisation.

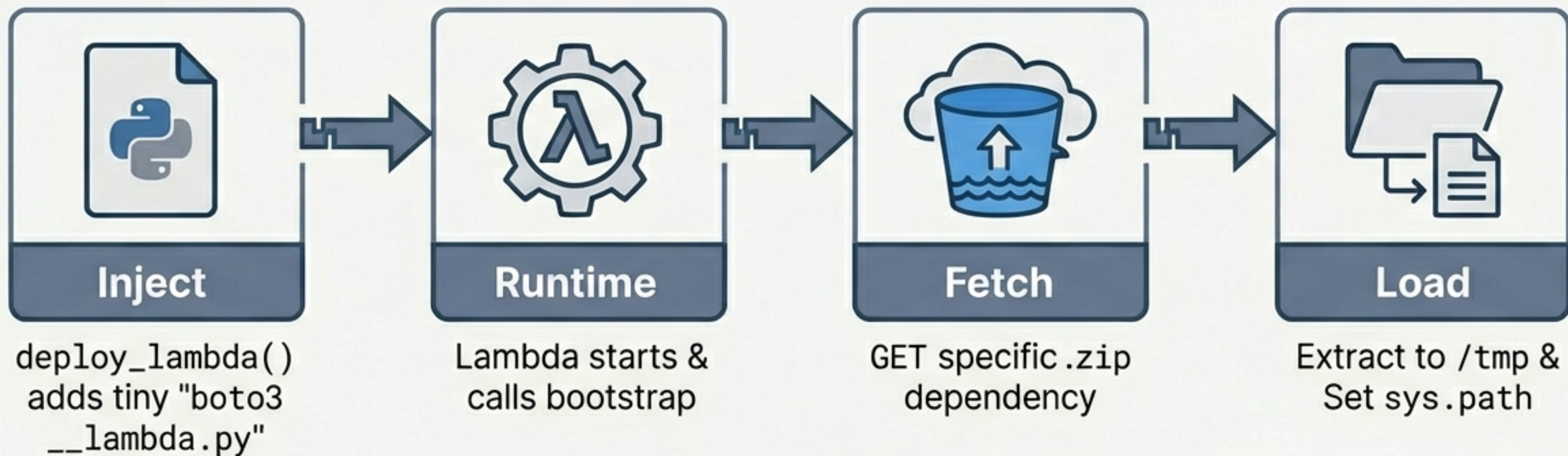
The Lambda Dependency Challenge

Why standard AWS Layers fail at scale.



How do we load dependencies like 'osbot-fast-api-serverless' **without bloated deployment packages** or restrictive layer limits?

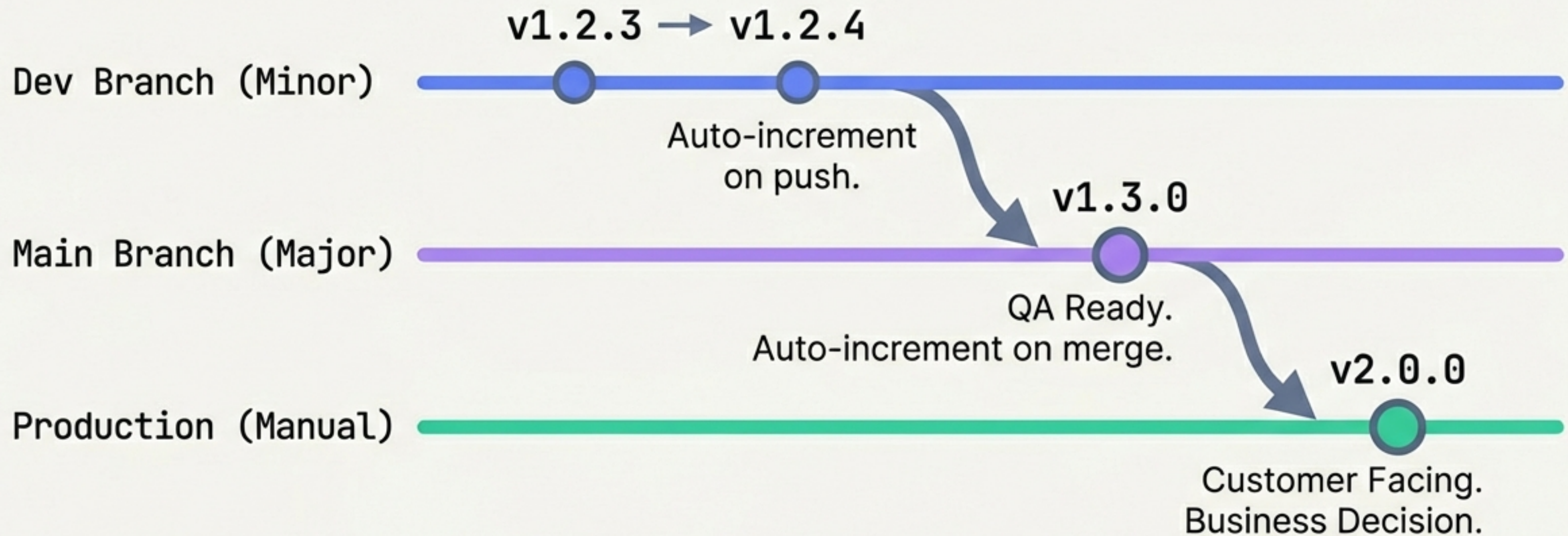
The Zero-Dependency Bootstrap Solution



Result:

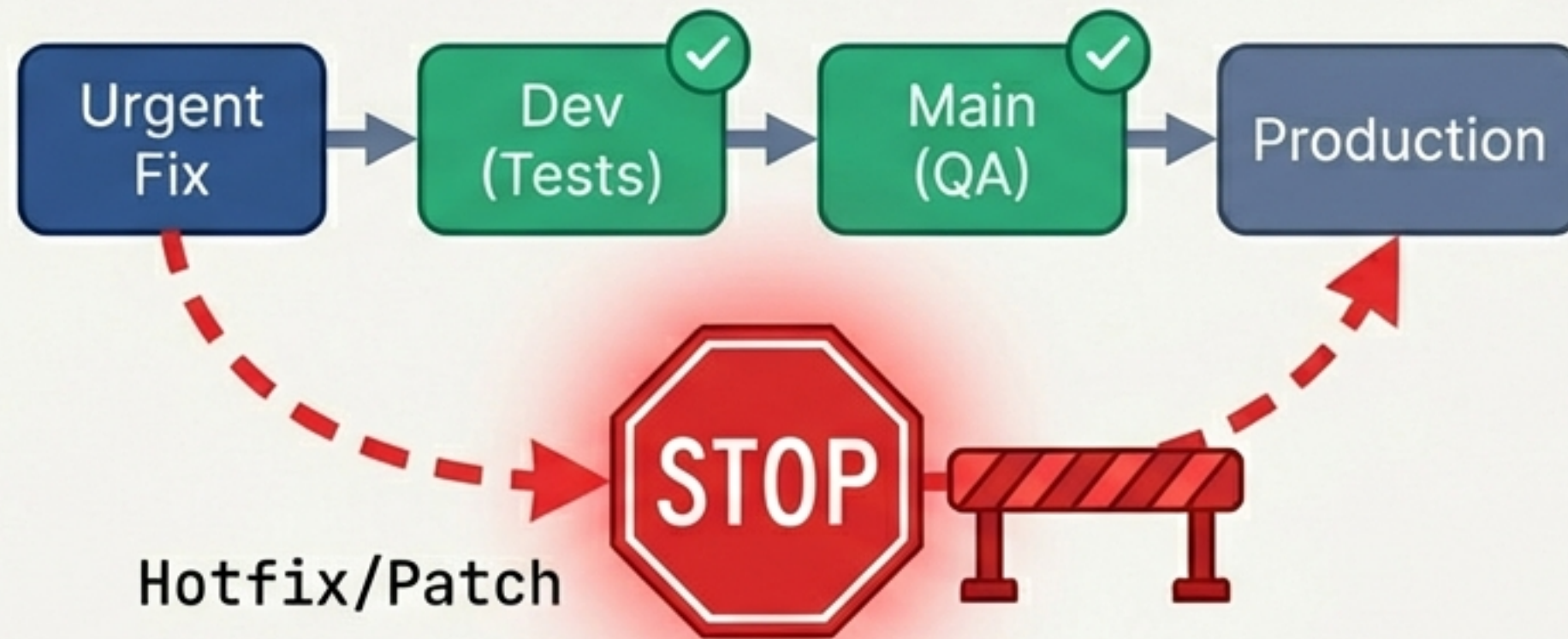
- Unlimited dependencies.
- Updates without redeploying the function.
- Faster cold starts.

Automated Governance: The Git Tagging Workflow



The pipeline dictates the version number, not the human.

Release Semantics: The "No Hotfix" Philosophy



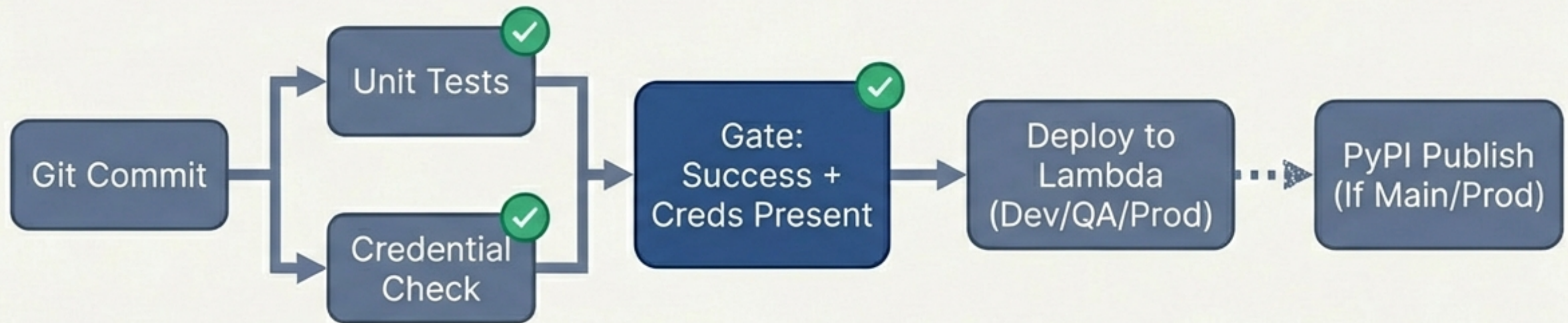
There is no concept of a "patch" release.

Principle: Integrity of testing over speed of hacking.

Rule: If **tests pass**, code is **ready for production**.

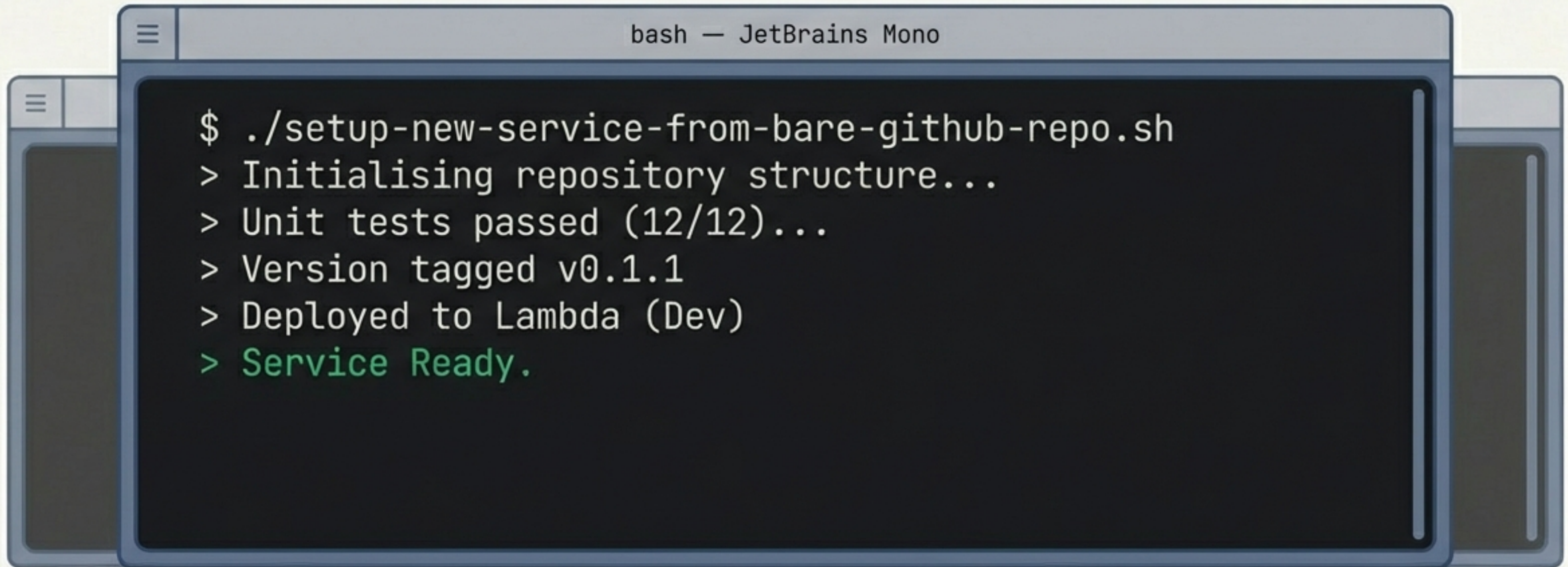
Every production release is simply a renamed, identical copy of a fully tested **Major release**.

Multi-Stage CI/CD Pipeline



Graceful handling of missing AWS credentials allows for local-only development without breaking the pipeline.

Bootstrapping a New Service

A terminal window with a dark background and light blue borders. The title bar reads 'bash — JetBrains Mono'. The terminal shows the execution of a script that initializes a repository, runs unit tests, tags a version, and deploys to Lambda. The final status 'Service Ready.' is highlighted in green.

```
bash — JetBrains Mono

$ ./setup-new-service-from-bare-github-repo.sh
> Initialising repository structure...
> Unit tests passed (12/12)...
> Version tagged v0.1.1
> Deployed to Lambda (Dev)
> Service Ready.
```

A reproducible, auditable foundation created in minutes.

Operational Safety & Configuration



Instant Rollbacks

Trivial rollback to any previous git tag via the pipeline.



Traceability

Every deployment is cryptographically linked to a specific commit hash.



Config-Driven

Behavior controlled via 'pyproject.toml' and strict environment variables.

Quantified Benefits

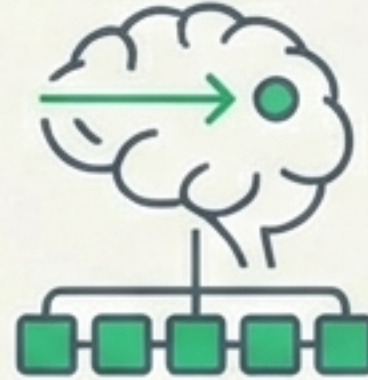
**2-4
Weeks**



Saved Per Project Project

Infrastructure setup
time eliminated.
Guaranteed test
coverage from day one.

0



Context Switching Load

Identical structure
across all services
reduces cognitive load
for developers.

100%

Audit Traceability Traceability

Every production artifact
is traceable to
source code
and approval.



The Infrastructure-First Philosophy

MGraph-AI Service Base is not just a template; it is a methodology.

By front-loading operational concerns, we allow engineers to focus entirely on business logic.

**“Focus on building features.
We’ve handled the rest.”**

